ГУАП

КАФЕДРА №

СОСТАВИТЕЛЬ

должность, уч. степень, звание	подпись, дата	инициалы, фамилия

МЕТОДИЧЕСКИЕ УКАЗАНИЯ К ПРАКТИЧЕСКОЙ РАБОТЕ № 2 МОДУЛЬНАЯ АРХИТЕКТУРА И КОНТРАКТ ПРОГРАММНОГО МОДУЛЯ

по МДК.01.04 «Системное программирование»

Санкт-Петербург

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 4

1 Цель и задачи работы 5

2 Теоретические сведения 6

2.1 Основные понятия 6

2.2 Контракт модуля 6

2.3 Связность, зацепление и направление зависимостей 7

2.4 Подмена реализации 7

3 Объект работы 9

4 Порядок выполнения работы 10

4.1 Выделение ответственности 10

4.2 Определение контракта 10

4.3 Реализация и интеграция 10

4.4 Подмена реализации 11

4.5 Модульная карта 11

4.6 Направления системной сложности 12

4.7 Модуль-кандидат и ADR 12

4.8 Проверки и взаимная проверка 13

4.9 Фиксация результата 13

5 Содержание отчёта 14

6 Критерии оценивания 15

7 Порядок защиты 17

8 Контрольные вопросы 18

ПРИЛОЖЕНИЕ А Пример оформления отчёта 19

Сведения о работе 19

1 Цель и задание 19

2 Проект и место модуля 19

3 Ход работы 20

3.1 Контракт (P2) 20

3.2 Реализация и интеграция (P3) 20

3.3 Подмена реализации (P4) 21

3.4 Модуль на карте (P5) 21

3.5 Профиль сложности, модуль-кандидат и ADR (P8) 21

4 Результаты 22

4.1 Проверки модуля (P6) 22

4.2 Взаимная проверка (P7) 22

5 Заключение 23

Список использованных материалов 23

ВВЕДЕНИЕ

Практическая работа № 2. Модульная архитектура и контракт программного модуля.

Компетенции: работа формирует ПК 1.2, ОК 01, ОК 04. Для студента, реализовавшего модуль или его существенную часть по контракту, работа является прямым подтверждением ПК 1.2; участие только в документации, тестах или инфраструктуре прямым подтверждением ПК 1.2 не является. Результаты работы учитываются в текущем контроле.

Форма: каждый студент выделяет модуль в своей зоне ответственности, определяет и документирует его контракт, реализует и интегрирует модуль, вносит его на модульную карту проекта, выбирает для своей зоны направления системной сложности и модуль-кандидат, оформляет собственный ADR и проверяет модуль другого студента. Отчёт оформляется по шаблону; защита и отметка индивидуальные.

Учебный пример отчёта VKP приведён в приложении А.

1 Цель и задачи работы

Цель работы — формирование умений выделять программный модуль из требований, определять его контракт и реализовывать модуль так, чтобы его реализацию можно было заменить без изменения потребителя.

Задачи работы:

1) Выделить из требований ответственность модуля и отделить её от внешних обязанностей.
2) Определить контракт модуля: входы, результат, ошибки, владение данными, побочные эффекты.
3) Реализовать модуль и подключить его к потребителю в проекте.
4) Подтвердить замену реализации без изменения потребителя.
5) Построить модульную карту продукта и проверить её соответствие коду.
6) Выбрать для своей зоны ответственности не менее двух направлений системной сложности и модуль-кандидат для последующих исследований; обосновать выбор в собственном ADR.

2 Теоретические сведения

2.1 Основные понятия

Таблица 1 — Термин / Определение

Термин	Определение
Модуль	Часть программы с определённой ответственностью и публичной границей: функция, класс, пакет, библиотека
Контракт	Наблюдаемые правила взаимодействия с модулем: допустимые входы, результаты, ошибки, гарантии
Потребитель	Код, вызывающий модуль через его публичный контракт
Подмена реализации	Подключение другой реализации за той же границей без изменения кода потребителя
Точка сборки	Место, где выбирается и связывается конкретная реализация
Тестовая подмена	Реализация с ограниченным поведением для проверки потребителя без реальной зависимости
Модульная карта	Схема модулей продукта с их ответственностями, контрактами и зависимостями
Модуль-кандидат	Существующий модуль, пригодный для отдельного наблюдения и исследования

2.2 Контракт модуля

Контракт описывает поведение модуля, а не только сигнатуру функции.

Таблица 2 — Часть контракта / Содержание

Часть контракта	Содержание
Назначение	Какую задачу решает модуль, публичная операция
Вход	Типы, диапазоны, формат, предусловия
Результат	Тип и смысл результата, гарантии (постусловия)
Ошибки	Категории ошибок, способ передачи, порядок проверки при нескольких нарушениях
Владение	Изменяется ли вход, кто владеет результатом, как долго действительны ссылки
Побочные эффекты	Изменение состояния, ввод-вывод, обращения к внешней среде

Одинаковые сигнатуры двух реализаций не означают их взаимозаменяемости, если они различаются смыслом результата, набором ошибок или владением данными.

2.3 Связность, зацепление и направление зависимостей

Модуль с высокой связностью решает одну задачу; низкое зацепление означает, что модули взаимодействуют только через публичные контракты и не зависят от внутреннего устройства друг друга. Запись A → B на модульной карте означает, что исходный код A зависит от контракта или типов B. Направление вызова при этом может не совпадать с направлением зависимости: потребитель вызывает реализацию, но в исходном коде зависит только от её контракта, а конкретная реализация выбирается в точке сборки.

2.4 Подмена реализации

Подмена возможна, если потребитель зависит от контракта, а не от конкретной реализации. Реализация передаётся потребителю через параметр, конструктор, фабрику или конфигурацию сборки. Например, модуль формирования отчёта получает источник данных через контракт «получить список записей»; в рабочей программе подключается источник из базы

данных, в тестах — фиксированный список. Код модуля отчёта при этом не меняется.

Тестовая подмена подтверждает работу потребителя в заявленном сценарии, но не исправность реальной зависимости — базы данных, сети, драйвера.

3 Объект работы

Объектом работы является программный модуль из зоны ответственности студента, реализующий содержательное правило продукта: проверку и преобразование данных, вычисление, правило выбора, управление состоянием. Модуль должен вызываться потребителем в проекте — сценарием каркаса, обработчиком, другим модулем или учебным потребителем. Одного интерфейса, заглушки или схемы недостаточно.

Модульная карта — общий артефакт проекта: каждый студент вносит на неё запись своего модуля и указывается её автором. Профиль системной сложности, модуль-кандидат и ADR каждый студент выполняет для своей зоны сам; направления и модули-кандидаты студентов одного проекта могут совпадать, но объекты, вопросы, обоснования и выводы у каждого свои. Искусственно делить архитектуру по числу студентов не требуется: несколько студентов могут работать с одним модулем, если каждый реализует собственную существенную часть. Студент, выполняющий проект один, составляет всю модульную карту сам, а взаимную проверку проводит с модулем студента другого проекта по назначению преподавателя.

4 Порядок выполнения работы

4.1 Выделение ответственности

Студентом выбирается часть полезной логики своей зоны и указываются реализуемые ею требования, потребитель и возможные причины изменения. Ответственность модуля отделяется от внешних обязанностей: например, правила обработки данных отделяются от чтения файла, интерфейса, сети и конкретного драйвера. Результат записывается цепочкой «требование → ответственность → публичная операция → потребитель».

4.2 Определение контракта

Контракт определяется по разделам п. 2.2 до демонстрации реализации и хранится рядом с кодом: в документации публичного API или отдельном файле. Если возможны одновременно несколько нарушений входа, определяется порядок их проверки. Отсутствие состояния или побочных эффектов указывается явно.

Документация на модуль оформляется по стандарту документирования кода, принятому для стека и указанному в декларации инструментального контура (например, docstring, Javadoc или KDoc, XML-комментарии C#, Doxygen, JSDoc или TSDoc). Она содержит назначение модуля, контракт публичной операции (вход, результат, ошибки), зависимости и пример использования. Документация фиксируется снимком экрана (рисунок 2).

4.3 Реализация и интеграция

Модуль реализуется и вносится по регламенту репозитория: задача, рабочая ветка, запрос на слияние, применимые проверки, независимая проверка, слияние. Модуль подключается к потребителю в проекте; потребитель не должен зависеть от внутренних структур модуля. Работоспособность каркаса практической работы №1 (сборка, версия, конфигурация, диагностика) сохраняется.

Публичная часть контракта и существенный фрагмент реализации фиксируются снимком экрана (рисунок 1). Запрос на слияние приводится в отчёте ссылкой; его содержимое в отчёт не копируется.

4.4 Подмена реализации

Подготавливается вторая реализация за той же границей: другая содержательная реализация модуля или тестовая подмена его зависимости с явно указанными ограничениями. Подмена подтверждается в следующем порядке:

1) Указываются потребитель и сценарий.
2) Обе реализации подключаются через точку сборки: параметр, конструктор, фабрику или конфигурацию.
3) Один и тот же сценарий выполняется с каждой реализацией.
4) Подтверждается, что код потребителя не изменялся: обе реализации вызываются из одного потребителя или приводится сравнение файлов потребителя.
5) Записываются результаты и границы вывода.

Переписывание потребителя под вторую реализацию подменой не является. Подключение двух реализаций к одному потребителю фиксируется снимком экрана (рисунок 3).

4.5 Модульная карта

Карта основных модулей продукта ведётся в репозитории как общий артефакт проекта: имя, ответственность, контракт, зависимости, место в репозитории, статус («работает» или «план»), автор записи. Стрелки обозначают зависимости исходного кода в смысле п. 2.3. Студент вносит или уточняет запись своего модуля и проверяет соответствие карты фактическим зависимостям своего модуля (подключаемые файлы, импорт, зависимости целей сборки); найденные циклические зависимости объясняются. Карта фиксируется снимком экрана (рисунок 4).

4.6 Направления системной сложности

Каждый студент для своей зоны ответственности предварительно выбирает не менее двух направлений, которые присутствуют в архитектуре продукта и затрагивают его участок. По ним в последующих работах студент выполняет собственные инженерные исследования. Направления студентов одного проекта могут совпадать; объекты, вопросы и выводы у каждого свои.

Таблица 3 — Направление / Примеры содержания

Направление	Примеры содержания
Модель исполнения	Событийная, последовательная или асинхронная организация и её последствия
Межкомпонентное взаимодействие	Протоколы, сообщения, границы процессов, отказы взаимодействия
Хранение и обработка данных	Проверка, преобразование, представление и объём данных
Интеграция с ОС и платформой	Файлы, сеть, устройства, службы, права доступа
Организация вычислений и памяти	Размещение данных, выделения памяти, копирование, алгоритмы
Аппаратно-специализированное исполнение	Использование специализированных возможностей процессора или ускорителя

Для каждого выбранного направления указываются объект на своём участке, исследовательский вопрос и текущая готовность. Выбор может быть пересмотрен до контрольной точки MVP с обоснованием. Название языка или библиотеки направлением не является.

4.7 Модуль-кандидат и ADR

Студент выбирает в своей зоне существующий модуль, которому можно подать воспроизводимый вход и наблюдать его поведение отдельно от остальной системы. Если подходящего модуля в зоне нет, выбирается участок общего модуля, с которым студент работает, с обоснованием выбора. Указывается будущий исследовательский вопрос; объявлять модуль узким местом до измерений не допускается.

Решение о границе модуля-кандидата студент оформляет собственным ADR с указанием себя автором: контекст, рассмотренные варианты, решение, последствия (в том числе цена решения — дополнительные типы, копирование данных, косвенные вызовы) и способ проверки.

4.8 Проверки и взаимная проверка

Для модуля выполняются проверки штатного случая, граничного значения и ошибочного входа с обеими реализациями. Результаты фиксируются снимком экрана (рисунок 5).

Студент проверяет модуль другого студента своего проекта; в одиночном проекте — модуль студента другого проекта по назначению преподавателя. Проверяющий получает зафиксированную версию, выполняет сборку по инструкции, проверяет случаи контракта и подмену, сверяет карту с зависимостями. Протокол содержит проверяющего, дату, commit, команды, результаты и замечания; при наличии замечаний после исправления проверка повторяется. Результат фиксируется снимком экрана (рисунок 6).

4.9 Фиксация результата

Сдаваемая версия фиксируется неизменяемым тегом `0104-pz02-<код>-v1`, где `<код>` — короткий идентификатор студента; при доработке создаётся новый тег (`0104-pz02-<код>-v2`), прежний тег не переносится. Срок сдачи и доработки — по памятке «Индивидуальная работа и сдача». Измерение производительности и локализация узких мест выполняются в последующих работах.

5 Содержание отчёта

Отчёт оформляется по шаблону и содержит:

1) Сведения о работе: студент, проект, модуль, версия сдачи.
2) Цель и задание: требования, ответственность модуля, потребитель.
3) Краткое описание проекта со ссылкой на модульную карту.
4) Ход работы: контракт и документация, реализация и интеграция, подмена, место модуля на карте, профиль системной сложности своей зоны, модуль-кандидат и собственный ADR.
5) Результаты: проверки модуля, взаимная проверка.
6) Заключение.
7) Список использованных материалов.

Обязательные иллюстрации отчёта:

Таблица 4 — Рисунок / Содержание снимка экрана

Рисунок	Содержание снимка экрана
1	Публичный контракт и существенный фрагмент реализации модуля
2	Документация на модуль по стандарту документирования стека
3	Один потребитель и подключение двух реализаций
4	Модульная карта с легендой и статусами
5	Результаты проверок модуля с обеими реализациями
6	Взаимная проверка модуля другого студента

6 Критерии оценивания

Таблица 5 — № / Обязательный результат

№	Обязательный результат	Критерий достаточности
Р1	Ответственность модуля	Цепочка «требование → ответственность → операция → потребитель»; внешние обязанности отделены
Р2	Контракт и документация	Определены вход, результат, ошибки и порядок их проверки, владение, побочные эффекты; документация на модуль оформлена по стандарту стека; рисунок 2
Р3	Реализация и интеграция	Модуль содержит реализованные правила и вызывается потребителем; внесён по регламенту, ссылка на запрос на слияние; рисунок 1
Р4	Подмена реализации	Один потребитель, две реализации, код потребителя не изменён, ограничения указаны; рисунок 3
Р5	Модуль на карте	Зависимости модуля на карте соответствуют коду; рисунок 4
Р6	Проверки модуля	Штатный, граничный и ошибочный случаи для обеих реализаций; рисунок 5
Р7	Взаимная проверка	Протокол проверки модуля другого студента (в одиночном проекте — студента другого проекта); рисунок 6
Р8	Профиль, модуль-кандидат и ADR	Для своей зоны — не менее двух направлений с объектом, вопросом и готовностью; модуль-кандидат с воспроизводимым входом; собственный ADR с автором, вариантами, решением, ценой и способом проверки

Таблица 6 — Оценка / Требования

Оценка	Требования
«Отлично»	P1–P8 выполнены полностью. Контракт описывает поведение, а не только сигнатуру; подмена подтверждена без изменения потребителя; ограничения тестовой подмены указаны. На защите действие выполнено самостоятельно
«Хорошо»	P1–P8 выполнены с несущественными недочётами в описании контракта, документации, карты или ADR. На защите потребовались уточняющие вопросы
«Удовлетворительно»	P1–P8 выполнены на минимальном уровне, контракт описан неполно. На защите потребовались наводящие вопросы
«Неудовлетворительно»	Обязательный результат отсутствует и не представлен в срок доработки; либо представлена чужая работа

Типичными основаниями для возврата работы являются: предъявлен только интерфейс без логики; модуль не вызывается потребителем; подмена потребовала переписать потребителя; реализации различаются набором ошибок; карта не соответствует коду; документация отсутствует или не описывает контракт; планируемые части показаны как работающие; модуль-кандидат существует только на схеме; профиль или ADR заимствованы у другого студента без собственного обоснования.

7 Порядок защиты

На защите студент выполняет одно-два действия по выбору преподавателя:

1) Показать публичную операцию модуля и объяснить один ошибочный исход.
2) Указать, какая часть реализации скрыта от потребителя.
3) Продемонстрировать подмену реализации.
4) Объяснить направление одной из стрелок модульной карты.
5) Изменить входные данные и предсказать результат по контракту.
6) Обосновать выбор модуля-кандидата и решение своего ADR.

При выполнении действий на защите генеративный ИИ не используется. Если студент не может выполнить действие или объяснить свой результат, работа возвращается на доработку: результат не засчитывается до повторной проверки, при повторной сдаче действие выполняется заново. Доработка выполняется в течение двух недель после возврата работы и представляется новым тегом; оценка после доработки не ограничивается.

8 Контрольные вопросы

1) Что такое контракт модуля и чем он отличается от сигнатуры функции?
2) Почему одинаковые сигнатуры не гарантируют взаимозаменяемости реализаций?
3) Что означают связность и зацепление модулей?
4) Чем направление зависимости отличается от направления вызова?
5) Какими способами реализация передаётся потребителю?
6) Что подтверждает и чего не подтверждает тестовая подмена?
7) Какие требования предъявляются к модулю-кандидату?
8) Какие разделы содержит ADR?

ПРИЛОЖЕНИЕ А

Пример оформления отчёта

Статус примера. Приведён учебный пример отчёта студента Б по проекту Vulkan Playground (VKP). Код не реализовывался и не запускался: контракт и решения являются учебной моделью, фактические результаты имеют статус «не выполнялось», места для снимков экрана не заполнены.

Сведения о работе

Таблица А.1 — Поле / Значение

Поле	Значение
Студент, группа	Б (учебный пример), `<группа>`
Проект	Vulkan Playground (VKP), три студента
Модуль	`DeviceSelection` — выбор видеокарты по настройке
Версия сдачи	`0104-pz02-b-v1` → `<commit>`

1 Цель и задание

Цель работы: формирование умений выделять программный модуль из требований, определять его контракт и реализовывать модуль так, чтобы его реализацию можно было заменить без изменения потребителя.

Ответственность модуля: требования FR-02 и FR-02.1 → выбор видеокарты из списка по значению настройки → операция `select(devices, value)` → потребитель `StartupScenario` (сценарий запуска приложения). Вне границы модуля: получение списка видеокарт от Vulkan, чтение переменной окружения, вывод сообщений.

2 Проект и место модуля

VKP — учебное приложение на C++ с использованием Vulkan. Модульная карта — общий артефакт проекта; запись `DeviceSelection` внесена студентом Б: `<ссылка, версия>`.

3 Ход работы

3.1 Контракт (P2)

Таблица А.2 — Часть контракта / Решение

Часть контракта	Решение
Операция	`select(devices, value)` — выбрать видеокарту по значению настройки
Вход	Список видеокарт (имя, тип); строка значения или признак её отсутствия
Результат	Номер и имя выбранной видеокарты
Ошибки и порядок проверки	`NoDevices` (список пуст) → `InvalidFormat` (значение не из цифр) → `IndexOutOfRange` (номер не меньше длины списка)
Владение	Вход не изменяется; результат содержит собственную копию имени
Побочные эффекты	Отсутствуют: нет вывода, обращений к Vulkan и файлам

3.2 Реализация и интеграция (P3)

Модуль реализован в `<путь>`. Документация оформлена комментариями Doxygen в заголовочном файле: назначение, операция `select`, коды ошибок, пример вызова. `StartupScenario` получает список видеокарт через контракт `DeviceProvider` (операция `list()`), вызывает `select` и передаёт результат дальнейшей инициализации. Изменение внесено запросом на слияние `<ссылка>`, проверено студентом В.

[МЕСТО ДЛЯ СКРИНШОТА А.1 — контракт DeviceSelection и фрагмент реализации]

Рисунок А.1 — Контракт и реализация модуля выбора видеокарты

[МЕСТО ДЛЯ СКРИНШОТА А.2 — документация DeviceSelection в Doxygen]

Рисунок А.2 — Документация на модуль выбора видеокарты

3.3 Подмена реализации (P4)

Подменяется зависимость потребителя — поставщик списка видеокарт:

Таблица А.3 — Реализация поставщика / Назначение

Реализация поставщика	Назначение	Ограничение
`VulkanDeviceProvider`	Получает список видеокарт через Vulkan	Требует драйвер и видеокарту
`FixedDeviceProvider`	Возвращает список из файла конфигурации	Не проверяет работу Vulkan и драйвера

Обе реализации передаются одному `StartupScenario` через конструктор в точке сборки `main`; выбор определяется параметром запуска `--devices=<файл>`. Код `StartupScenario` при подмене не изменяется.

[МЕСТО ДЛЯ СКРИНШОТА А.3 — точка сборки и запуск StartupScenario с двумя поставщиками]

Рисунок А.3 — Подключение двух реализаций к одному потребителю

3.4 Модуль на карте (P5)

Зависимости исходного кода: `StartupScenario → DeviceSelection`, `StartupScenario → DeviceProvider`, `VulkanDeviceProvider → DeviceProvider`, `FixedDeviceProvider → DeviceProvider`. Точка сборки `main` знает конкретные реализации. `DeviceSelection` ни от чего, кроме стандартной библиотеки, не зависит. Соответствие проверено по подключаемым заголовочным файлам.

[МЕСТО ДЛЯ СКРИНШОТА А.4 — модульная карта VKP]

Рисунок А.4 — Модульная карта продукта

3.5 Профиль сложности, модуль-кандидат и ADR (P8)

Направления для зоны студента Б: интеграция с ОС и платформой (загрузка Vulkan, драйвер, отказы среды) и хранение и обработка данных (будущая подготовка геометрии модели). Модуль-кандидат — инициализация Vulkan в `VulkanDeviceProvider`: её можно запускать отдельно и наблюдать время создания экземпляра Vulkan. Будущий вопрос: как включение слоёв

проверки Vulkan влияет на время запуска. ADR-002 (автор — студент Б) фиксирует выделение `DeviceProvider`: рассмотрены вызов Vulkan прямо из сценария запуска и выделение поставщика; выбран поставщик; цена решения — дополнительный интерфейс; способ проверки — запуск сценария с двумя поставщиками.

4 Результаты

4.1 Проверки модуля (P6)

Таблица А.4 — Случай / Ожидаемый результат

Случай	Ожидаемый результат	Фактический результат
Две видеокарты, значение «1»	Видеокарта 1	Не выполнялось
Две видеокарты, значение отсутствует	Видеокарта 0	Не выполнялось
Две видеокарты, значение «2»	`IndexOutOfRange`	Не выполнялось
Пустой список, значение «zero»	`NoDevices` (проверяется первой)	Не выполнялось
`StartupScenario` с `FixedDeviceProvider` и с `VulkanDeviceProvider`	Одинаковое поведение выбора при одинаковом списке	Не выполнялось

[МЕСТО ДЛЯ СКРИНШОТА А.5 — результаты проверок с обоими поставщиками]

Рисунок А.5 — Проверки модуля выбора видеокарты

4.2 Взаимная проверка (P7)

Студент Б проверяет модуль студента В `<модуль>` по версии `<commit>`: сборка по инструкции, случаи контракта, подмена, соответствие карты. Протокол: `<ссылка>`.

[МЕСТО ДЛЯ СКРИНШОТА А.6 — проверка модуля другого студента]

Рисунок А.6 — Взаимная проверка

5 Заключение

Заключение формулируется по фактическим результатам. Пример формулировки: «Реализован модуль выбора видеокарты с контрактом, определяющим порядок проверки ошибок. Подмена поставщика списка видеокарт выполнена без изменения сценария запуска, что позволяет проверять правило выбора на компьютере без видеокарты. Фиксированный поставщик не подтверждает работу драйвера; она проверяется запуском с `VulkanDeviceProvider`».

Список использованных материалов

1) Спецификация требований VKP, FR-02 и FR-02.1 — `<ссылка, версия>`.
2) Модульная карта проекта, профиль системной сложности и ADR-002 студента Б — `<ссылки, версии>`.
3) Запрос на слияние студента Б — `<ссылка>`.
4) Протокол взаимной проверки — `<ссылка>`.